This ICCV paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

FedPerfix: Towards Partial Model Personalization of Vision Transformers in Federated Learning

Guangyu Sun[1], Matias Mendieta[1], Jun Luo[2], Shandong Wu[3], Chen Chen[1]

[1]Center for Research in Computer Vision, University of Central Florida, USA
[2]Intelligent Systems Program, University of Pittsburgh, USA
[3]Department of Radiology, Biomedical Informatics,
and Bioengineering, University of Pittsburgh, USA

{guangyu.sun, matias.mendieta}@ucf.edu; jul117@pitt.edu;
wus3@upmc.edu; chen.chen@crcv.ucf.edu

Abstract

Personalized Federated Learning (PFL) represents a promising solution for decentralized learning in heterogeneous data environments. Partial model personalization has been proposed to improve the efficiency of PFL by selectively updating local model parameters instead of aggregating all of them. However, previous work on partial model personalization has mainly focused on Convolutional Neural Networks (CNNs), leaving a gap in understanding how it can be applied to other popular models such as Vision Transformers (ViTs). In this work, we investigate where and how to partially personalize a ViT model. Specifically, we empirically evaluate the sensitivity to data distribution of each type of layer. Based on the insights that the self-attention layer and the classification head are the most sensitive parts of a ViT, we propose a novel approach called FedPerfix, which leverages plugins to transfer information from the aggregated model to the local client as a personalization. Finally, we evaluate the proposed approach on CIFAR-100, OrganAMNIST, and Office-Home datasets and demonstrate its effectiveness in improving the model's performance compared to several advanced PFL methods. Code is available at https://github.com/imguangyu/FedPerfix

1. Introduction

Federated learning (FL) [25] has emerged as a promising method for training machine learning models on decentralized data without requiring direct data sharing. However, data heterogeneity among participating clients can present a significant challenge. Due to the various circumstances of the clients, the data across the clients can be non-independent and non-identically distributed (non-IID). Therefore, achieving satisfactory performance using a one-model-fits-all approach is difficult, and personalized models are often needed to achieve the best results. This has inspired the study of Personalized Federated Learning (PFL), where the focus is shifted from the performance of the global model on the server to the local models on the clients.

In the context of personalized federated learning, previous literature has explored two main approaches: *full model personalization* [9, 6, 23] and *partial model personalization* [31, 27, 18, 5, 1]. Full model personalization involves maintaining a separate local model for each client and updating it based on a joint objective, while partial model personalization aims to personalize only a subset of the model parameters. A convergence analysis [31] suggests that partial model personalization can achieve most of the benefits of full model personalization with fewer shared parameters, offering advantages in terms of computation, communication, and privacy to enable the deployment of larger models on the clients.

However, recent literature shows that *where* and *how* to perform partial model personalization has a high correlation to the model architectures and the tasks [31], which requires further study when applied to a new architecture. Despite the multitude of approaches proposed in the literature, the majority of methods have only been evaluated on Convolutional Neural Networks (CNNs). Meanwhile, Vision Transformers (ViTs) [8] have demonstrated superior performance compared to CNNs in several tasks, such as image classification [37] and object detection [42, 3], making them an attractive option for personalized federated learning. However, to the best of our knowledge, the application of ViT in the federated learning community has received limited at-

tention in the existing literature [32]. Given the advantages of ViT shown under centralized training, it is reasonable to expect that these benefits can also be realized in PFL by offering a more robust model for improved performance on the clients. Therefore, in this work, we investigate *where* and *how* to *partially personalize* a *ViT* model.

Drawing from previous research on CNNs, layers that serve specific engineering purposes, such as feature extraction, normalization, or classification, have been identified as suitable candidates for partial model personalization [27, 18, 5]. These layers might have a higher *sensitivity* to the distribution of the training data. Therefore, aggregating the model weights trained from different data distributions may result in an inaccurate feature, while keeping them updated locally can gain a better feature for the local data distribution. Similarly, we select some candidates from ViT and conduct an empirical study to investigate the sensitivity of each type of layer. Specifically, we quantitatively evaluate the impact of keeping certain layers updated locally without aggregation with other clients. This evaluation shows that the *self-attention layers* and the *classification head* have a higher sensitivity than other layers, providing insights about *where* to personalize.

To personalize the sensitive parts, one intuitive approach is to keep them completely local. However, the global aggregation has shown the capability to provide a more general global model than local models [24]. Completely preventing the sharing with the global model will severely hinder the potential benefits of leveraging general knowledge from the aggregated global model. Therefore, we desire to train a personalization module to bridge the general knowledge and the client-specific knowledge.

As existing works in transfer learning suggest, the same pre-trained model can be transferred to different downstream data by adding different tiny architectures [30, 15, 20, 12]. We refer to these tiny architectures as *plugins* since they are small-sized parameters plugged into the pre-trained model. These plugins are trained while the weights of the pre-trained model are frozen. The plugged model can achieve comparable performance as the fully fine-tuned model [29]. Since the model can maintain the same level of performance without changing the weights of the pre-trained model, the downstream-specific knowledge is captured by the plugins. Therefore, the plugins show the capability to *personalize* the same model to satisfy different downstream data. In federated learning, we can consider the aggregated model as an inferior version of the "pre-trained" model, and the data on different clients as the downstream data, as shown in Fig. 1. With this perspective, we can therefore exploit the capability of the plugins in the federated learning context to capture client-specific knowledge for personalization.

Therefore, we select and adapt a specific family of plugin, *Prefixes*, to personalize the self-attention layer and propose a novel approach **FedPerfix**, short for **Fed**erated **Per**sonalized Pre**fix**-tuning.

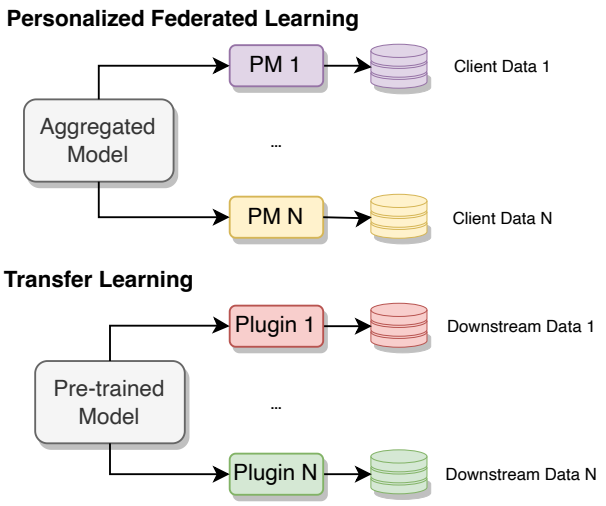


Figure 1. **Analogy between transfer learning and personalized federated learning.** A pre-trained model can be transferred to different downstream data with different plugins. In personalized federated learning, we are seeking different personalization modules (PM) to transfer the aggregated model to different client data.

The main contributions of this paper are as follows:

- We perform an empirical study to reveal the sensitivity to data distribution of each type of layer in a ViT for PFL and locate the self-attention layer and classification head as the sensitive part to be personalized. (Section 3.3)
- We propose a novel partial model personalization approach on ViT, FedPerfix, inspired by the connection between PFL and transfer learning. Specifically, we exploit Prefix plugins to capture client-specific knowledge for personalization. (Section 3.5)
- We conduct evaluations on CIFAR-100 [2], OrganAMINIST [40], and Office-home [38], which are across different domains, and degrees of data heterogeneity and achieve state-of-the-art performance compared with several competitive methods with lower resource requirements. (Section 4.2)

2. Related Work

Personalized Federated Learning. Personalized Federated Learning focuses on training a client-specific model to achieve satisfying performance on each client instead of a unified global model to fit all the client data. In terms of the shared parameters, full model personalization keeps a separate personalized model for each client and designs different training objectives, where the global model is usually served as the regularization term [36, 17, 43, 19, 13, 26].

The training objectives are designed with the idea of meta-learning [9], Moreau Envelopes [7], model interpolation [6], decoupling [4], *etc*. In contrast, partial model personalization focuses on personalizing some specific parameters inside a model. Some works simply keep these parameters, *e.g*., the classification head [1, 27], and the batch normalization layers [18] updated locally, while some works design additional parameters, *e.g*., prototypes for each class [36] or hypernetworks [34], to further personalized these parameters. Our proposed method focuses on the self-attention layers in a ViT and leverages plugins in transfer learning, enabling personalization without introducing significant overhead. Our experimental results demonstrate that FedPerfix outperforms existing methods in terms of both efficiency and effectiveness.

Vision Transformer. ViTs, a type of attention-based neural network, have shown superior performance in several fields compared to CNNs. In particular, ViTs have achieved state-of-the-art performance on various benchmarks [8, 22, 21]. ViTs employ self-attention mechanisms to learn the dependencies between different regions of an image and can capture long-range dependencies more effectively than CNNs. Despite their success, the use of ViTs in federated learning has not been as widely explored as CNNs. Qu *et al*. [32] studied the performance of ViT under conventional FL settings and found that simply replacing CNNs with ViTs can greatly accelerate convergence and reach a better global model, especially when dealing with heterogeneous data, demonstrating the potential of ViTs in FL. In this paper, we extend the application of ViT to PFL and propose a novel approach to better personalize a ViT.

Parameter-efficient Fine-tuning and Plugins. Fine-tuning is a popular technique for adapting a pre-trained model to a new task. However, fine-tuning can be computationally expensive, especially for large models [10, 41]. Parameter-efficient fine-tuning (PEFT) has been proposed to address this issue by selectively updating only a subset of the model's parameters or a few added parameters. These added parameters are plugged into the pre-trained model; therefore, they are summarised as "Plugins" in this paper. Although the location of the plugins for a transformer is various, including the input [15], the self-attention layer [20], and the feed-forward network [30], all of them can be trained to achieve comparable performance as fine-tuning without updating the weights of the pre-trained model. The plugins have also been applied to conventional FL to enable large pre-trained models and reduce the communication cost [35]. In the PFL, inspired by analogy from fine-tuning to the local training, the plugins can similarly be employed to personalize the representation from the global model to the local data distribution. Based on our empirical study, we choose prefixes [20] since the parameters are directly applied to the self-attention layer and propose our method, FedPerfix.

3. Method

3.1. Problem Formulation

We consider a classification problem in Computer Vision (CV). A dataset $\mathcal{D} = \{(x, y)|x \in X, y \in Y\}$ is separated on N clients, where X is the input space and Y is the label space. Data on each client is denoted as $\mathcal{D}_i$, and the distribution P_i. The client distributions are not identical. Each client has access to m_i samples drawn IID from the distribution P_i. The total number of samples is $M = \sum_{i=1}^{N} m_i$. The hypothesis (model) on each client is noted as h_i, and the expected loss on the ith client is denoted as

$$\mathcal{L}_{\mathcal{D}_i}(h_i) = \mathbb{E}_{(x,y)\sim P_i}[\ell(h(x), y)] \tag{1}$$

where ℓ is the loss function. Further, considering the *partial model personalization*, the parameters in each client model can be divided into two parts: global parameters u and local parameters v_i, i.e., $h_i = u \cup v_i$. Specifically, $v_i = \varnothing$ indicates *full model personalization*.

In each communication round $t \in [T]$, each client will receive $u^{(t-1)}$ from the server and plug $v_i^{(t-1)}$ into $u^{(t-1)}$. Then, $u_i^{(t)}, v_i^{(t)}$ are trained on the local data. After the local training, only the $u_i^{(t)}$ of K sampled clients will be sent to the server and participate in the global aggregation. The sampled ratio (client participation rate) is denoted as $r = K/N$. The global aggregation is formulated as

$$u^{(t)} = \sum_{i=1}^{K} \alpha_i u_i^{(t)} \tag{2}$$

where α_i is the aggregation coefficient. Federated Averaging (FedAVG) is the most common aggregation algorithm where $\alpha_i = \frac{m_i}{M}$. After T rounds of communication, each client model will receive a copy of the newest global parameters and plug its own local parameters into the global parameters. Then, the model will be evaluated on its own test data drawn IID from P_i. To assess the overall performance, the mean and standard deviation of the **Top-1 classification accuracy (Acc)** for all clients will be reported.

3.2. Recap: Partial Model Personalization for CNN

In this section, we provide a brief recap of the approaches for partial model personalization in CNNs. We aim to leverage the insights and experiences gained from previous research to explore the strategy to personalize ViTs partially.

FedRep [5] is a partial model personalization method that aims to preserve client-specific information while leveraging common knowledge in the earlier layers of a CNN by keeping the last classification layer and a few blocks local

Table 1. **Sensitivity to data distribution of each type of layer in a Vision Transformer.** The *mean* and *standard deviation* of the client's Top-1 Accuracy are reported for each type of layer (**Stand-alone**). Considering the classification head is the most sensitive type of layer, we also report the performance when a type of layer is kept updated locally along with the classification head (**Combined**). The **overall** performance is the mean of the stand-alone accuracy and combined accuracy.

Type	Stand-alone	Combined	Overall
All Local	$34.74_{\pm 9.36}$	-	-
All Global	$23.29_{\pm 11.31}$	-	-
Classification Head	$44.42_{\pm 7.98}$	-	-
Patch Embedding	$27.61_{\pm 10.02}$	$43.45_{\pm 8.69}$	35.53
Position Embedding	$23.80_{\pm 11.42}$	$45.04_{\pm 8.08}$	34.42
LayerNorm	$23.94_{\pm 11.19}$	$44.60_{\pm 7.96}$	34.27
Self-attention	$42.95_{\pm 8.68}$	$44.63_{\pm 8.67}$	**43.79**
MLP	$42.53_{\pm 8.82}$	$42.63_{\pm 8.84}$	42.58

and aggregating the rest. **FedBN [18]** is a partial model personalization method that updates Batch normalization (BN) [14] layers locally while aggregating the rest of the model. This method leverages the local statistics of the BN layer to adapt to the data distribution of each client while maintaining a global representation of the model, achieving a better balance between personalization and global representation. **FedBABU [27]** focuses on personalizing the classification head to address the issue of inconsistent feature spaces among clients. Unlike FedRep, FedBABU freezes the classification head and fine-tunes it for several steps before evaluation. By doing so, FedBABU can mitigate the negative impact of classification head drift and encourage consistent feature spaces across all clients.

In summary, previous approaches to partial model personalization for CNNs provide valuable insights to answer the "where to personalize" question for ViTs: The layers to be personalized are usually designed with some engineering purpose, thus, are sensitive to data distribution. For instance, the normalization layers are designed to capture the statistical characters of the data, while classification heads are designed to map the feature to a predicted class. Building on these insights, we group the layers in ViT by their engineering purpose and conduct an empirical study to determine the layers in ViT that are the most sensitive to data distribution. We evaluated the performance of ViT while keeping some specific types of layers updated locally without aggregating with other clients. The results of our study are presented in Section 3.3.

3.3. Empirical Study: Partial ViT Personalization

In a ViT, we consider the following layers as candidates to be personalized and evaluate their sensitivity to data distribution when they are updated locally: **Patch Embedding** layer converts the input image to a sequence of patches, then applies a linear projection to map each patch into an embedding vector. **Position Embedding** layer is responsible for injecting positional information into the sequence of patches produced by the patch embedding layer, allowing the model to capture the spatial arrangement of the image. **LayerNorm** is a normalization layer commonly used in ViT to normalize the feature maps across the channel dimension. **Self-attention layer** allows the model to attend to different regions of the input image and capture long-range dependencies between them. **MLP** layers, following the self-attention layer, enable the model to capture non-linear relationships between the image patches. **Classification Head** applies a linear projection from the [CLS] token to a predicted class.

We perform experiments on CIFAR-100 with our default setting, which will be introduced in Section 4.1. We keep the specified type of layer updated locally and report the mean and standard deviation of the Top-1 Accuracy across all clients as the metrics to indicate the *sensitivity* to data distribution. The result is shown in the second column of Table 1.

From the table, we can draw several insights. Keeping all layers local yields better performance than aggregating all layers. This suggests that aggregation of heterogeneous data may have a negative impact on some layers. Meanwhile, keeping each type of layer updated locally allows for greater adaptability to each client's data, leading to different degrees of improvement compared with keeping every layer global. Among all types of layers, the classification head is the most sensitive to data distribution, leading to the highest stand-alone performance, which is as expected. Except for the classification head, self-attention layer is more sensitive to data distribution than other components, showing a higher standard-alone performance.

Considering the vital importance of the classification head, we also reported the combined performance when a type of layer is kept updated locally along with the classification head, as shown in the third column of Table 1. To decide the most sensitive layers to be personalized, we averaged the performance under the two settings as the overall performance.

Based on the overall performance, personalizing the self-attention and classification head is one possible answer to the question of *where* to personalize. Besides, a vanilla baseline to keep the self-attention layers and classification head updated locally is proposed, which is referred to as Vanilla Attention as described in Fig. 2 (a).

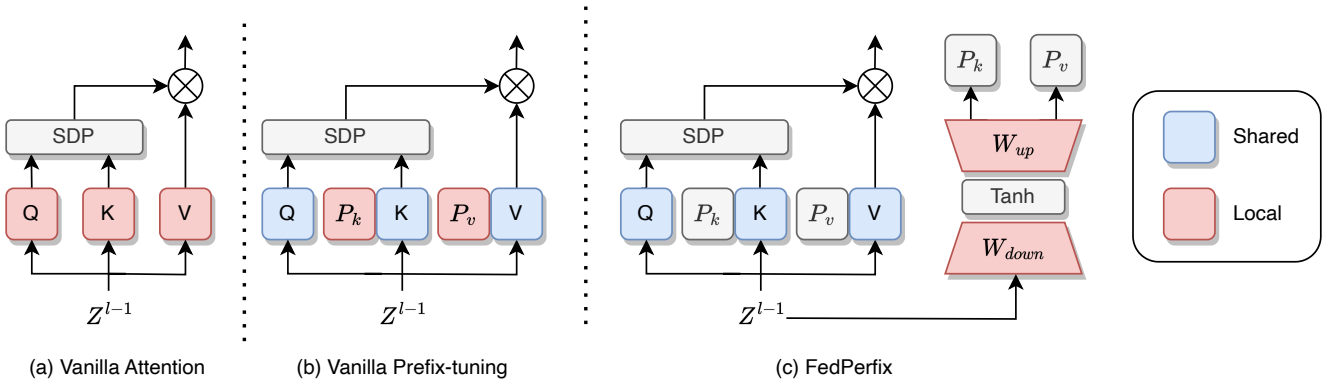


Figure 2. **Several designs to personalize the self-attention layer in a ViT.** SDP is short for Scaled Dot Product, the red part indicates the local parameters, the blue part indicates the global parameters and the gray parts are vectors or modules with no learnable parameter. (a) Vanilla attention simply keeps the self-attention and classification updated locally. (b) Vanilla prefix-tuning only keeps the prefixes updated locally and aggregates the original self-attention layer. (c) FedPerfix uses a local adapter to generate the prefixes and aggregates the original self-attention layer. The adapter is composed of a scale-down, an activation, and a scale-up layer.

3.4. Proposed Baseline: Vanilla Prefix-tuning

Building upon the insights about *where* to personalize gained in the previous section, we move to the next question about *how* to personalize the selected layers. As obtained from our empirical study, Vanilla Attention is a baseline to personalize the self-attention layer by completely keeping the self-attention updated locally. However, it will also prevent it from learning general information for the global model.

As explained in Section 1, the plugins can be trained on the local data to capture the client-specific knowledge as a personalization module. There are three widely used plugins for ViTs: a) **Prompts** that are learnable embeddings appended to the inputs, b) **Adapters** that are inserted into the MLP layers, and c) **Prefixes** that are appended to the key and value matrix of the self-attention layer. Given that the self-attention layer is one of the most sensitive layers, It is plausible that the Prefixes integrated nearest to the self-attention layer are capable of capturing the most relevant client-specific information pertaining to the self-attention mechanism. Besides, we further compare the effectiveness of all three plugins and discuss it in Section 5.3. Therefore, we select Prefixes as our personalization module.

Specifically, learnable prefixes are appended to the self-attention layer and kept updated locally on the client. Meanwhile, the original self-attention layer is shared across all clients as normal to capture global dependencies. We refer to this approach as Vanilla Prefix-tuning.

An illustration of how Prefixes cooperate with the self-attention layer is shown in Fig. 2 (b). Besides, we formulate the output of one head of the self-attention layer as

$$\begin{aligned} head_i = &Attention(\boldsymbol{Z}^{l-1}\boldsymbol{W}_q^{(i)}, \\ &\boldsymbol{Z}^{l-1}[\boldsymbol{P}_k, \boldsymbol{W}_k^{(i)}], \\ &\boldsymbol{Z}^{l-1}[\boldsymbol{P}_v, \boldsymbol{W}_v^{(i)})]) \end{aligned} \tag{3}$$

where $\boldsymbol{Z}^{l-1}$ is the output from the last layer, $\boldsymbol{P}_k$ and $\boldsymbol{P}_v$ are prefixes, $\boldsymbol{W}_q^{(i)}$, $\boldsymbol{W}_k^{(i)}$, and $\boldsymbol{W}_v^{(i)}$ are the parameters in the self-attention layer, and $[,]$ is the concatenate operation.

3.5. FedPerfix: Stabilize the Prefix

In Vanilla Prefix-tuning, the prefixes are randomly initialized, which can result in unstable performance when initialized with different weights. We later conduct an experiment to demonstrate such instability in Section 5.3 when Vanilla Prefix-tuning is initialized with different weights. To address this issue, we draw inspiration from the parallel attention design [41], which uses adapters to stabilize the prefixes. Similarly, we propose to use adapters in Vanilla Prefix-tuning to stabilize the prefix initialization. Specifically, we add a parallel adapter to prepare the prefixes for each layer, as shown in Fig. 2. Meanwhile, we add a hyper-parameter s to control the efficiency of the adapter. Following the design of parallel attention, the prefixes are generated as

$$\boldsymbol{P}_k, \boldsymbol{P}_v = Adapter(\boldsymbol{Z}^{l-1}) = \text{Tanh}(\boldsymbol{Z}^{l-1}\boldsymbol{W}_{down})\boldsymbol{W}_{up} \tag{4}$$

where Tanh is the activation function, $\boldsymbol{W}_{down}$ and $\boldsymbol{W}_{up}$ are parameters of the scaling layers of the adapter. As a result, the output of one head of the self-attention layer with paral-

lel attention is shown in Fig. 2(c) and formulated as

$$\begin{aligned} head_i = &Attention(\boldsymbol{Z}^{l-1}\boldsymbol{W}_q^{(i)}, \\ &\boldsymbol{Z}^{l-1}[s\boldsymbol{P}_k, \boldsymbol{W}_k^{(i)}], \\ &\boldsymbol{Z}^{l-1}[s\boldsymbol{P}_v, \boldsymbol{W}_v^{(i)}]). \end{aligned} \tag{5}$$

In conclusion, FedPerfix is proposed as a novel approach to transfer the information from the aggregated self-attention layer to fit the local data better, leveraging the plugins in transfer learning. The local prefixes are trained for personalization, while the global self-attention layer captures global dependencies. Therefore, FedPerfix provides a promising solution to address the question of how to perform personalization for ViTs.

4. Experiment

4.1. Experiment Details

Dataset. We use three popular datasets to evaluate the performance: **CIFAR-100** [16], **OrganAMNIST** [40], and **Office-Home** [38]. CIFAR-100 is a widely-used image classification dataset consisting of 50,000 RGB images across 100 classes. Office-Home is a domain adaptation dataset for object recognition tasks, which contains 15,500 images across 65 categories, captured from four different domains: Artistic images, Clipart, Product images, and Real-World images. OrganAMNIST is a medical image classification dataset that includes 58,850 gray-scale images of organs and tissues from human anatomy. The images are classified into 11 different classes. These datasets are chosen to evaluate the performance in various scenarios, including different data scales, domains, and the number of classes.

Federated learning settings. We conduct $T = 50$ communication rounds with each local training consisting of 10 epochs. To account for the various factors that can affect federated learning performance, we adjust the default settings for each dataset to focus on different scenarios with varying degrees of data heterogeneity of **label skew** and **concept skew**. For CIFAR-100, which has a relatively high number of samples and classes, we simulate a federated learning environment with $N = 64$ clients using a Dirichlet distribution with $\alpha = 0.1$ and sampling $K = 8$ clients with a ratio $r = 0.125$ for each round. For OrganAMNIST, which has a similar number of samples but fewer classes than CIFAR-100, we partition the data using a Dirichlet distribution with a larger $\alpha = 0.5$ and apply the same client settings as CIFAR-100. As for Office-Home, which has fewer samples captured from 4 different domains, we focus on concept skew by partitioning the data from each domain into four different clients using a Dirichlet distribution with $\alpha = 1.0$, resulting in a total of $N = 16$ clients. In each communication round, we randomly sample $K = 4$ clients from any domain with a ratio $r = 0.25$. A visualization of the data partitioning is provided in Supplementary A.1.

Model Architecture and Baselines. We choose ViT-Small (ViT-S) [8] with a patch size of 16 and an image size of 224 as our model for evaluation. We compare our proposed FedPerfix method against a range of baseline methods, including widely used approaches like FedAVG and local training without aggregation (Local). We also compare our method against advanced full model personalization methods, namely APFL [6] and Per-FedAVG [9], and three partial model personalization methods, namely FedBN, FedRep, and FedBABU. Besides, we also include the performance of Vanilla Attention, which keeps the self-attention layer and the classification head updated locally, as a reference. APFL adapts the global and local model through an adaptive mixture coefficient, while Per-FedAVG employs meta-learning to improve the global model. These methods are model-agnostic and can be applied to ViTs. FedBN keeps the batch normalization layer updated locally and aggregates the remaining layers in the server with FedAVG. To adapt it to ViTs, we replace the batch normalization layer with the layer normalization layer. FedRep keeps the classification head updated locally while aggregating the other layers in the server. FedBABU freezes the classification head and aggregates the remaining layers in the server, then fine-tunes the classification head on the local data for one step before the evaluation.

Evaluation Metrics. We report the final mean and standard deviation of Top-1 accuracy among all clients after all communication rounds finish as the evaluation metrics. Besides, to evaluate the feasibility of the methods, an analysis of storage, computation, and communication costs is reported in Section 4.4.

Implement Details. All hyperparameters in each method are tuned as optimal in a range. Each model is optimized with the SGD [33] optimizer with the optimal learning rate, which is 0.01 for most methods. The server and all the clients are simulated in one machine, and we resize the images in all datasets as 224×224 RGB images to fit the model and set the batch size of the client data as 64. The model is implemented from the TIMM [39] library. All the experiments are implemented in Pytorch [28] and performed on 4 Nvidia A5000 GPUs. More details are provided in Supplementary A.2.

4.2. Performance Evaluation

Based on the evaluation results presented in Table 2, several insights can be drawn regarding the performance of the compared methods. It can be observed that FedAVG performs worse than Local when facing a large label skew, *i.e.*, on CIFAR-100. This indicates the importance of addressing data heterogeneity in federated learning scenarios. However, when the label skew is not that extreme, the informa-

Table 2. **Performance and required resources for each method.** The mean and standard deviation of the Top-1 Accuracy among all clients are reported. The bold style indicates the best performance in each dataset.

Method	Performance			Storage	Computation	Communication
	CIFAR-100	OrganAMNIST	Office-Home	(# Params)	(FLOPs)	(# Params)
FedAvg	$23.29_{\pm 11.31}$	$87.31_{\pm 5.98}$	$21.47_{\pm 6.24}$	21.03M (100%)	65.65M (100%)	21.03M (100%)
Local	$34.74_{\pm 9.36}$	$78.26_{\pm 11.07}$	$20.39_{\pm 7.26}$	21.03M (100%)	65.65M (100%)	0 (0%)
APFL	$44.88_{\pm 10.50}$	$89.74_{\pm 5.83}$	$24.23_{\pm 7.02}$	42.06M (200%)	131.30M (200%)	21.03M (100%)
Per-FedAVG	$33.86_{\pm 8.01}$	$82.81_{\pm 7.13}$	$17.09_{\pm 4.83}$	21.03M (100%)	131.30M (200%)	21.03M (100%)
FedBN	$23.94_{\pm 11.19}$	$87.63_{\pm 5.78}$	$21.25_{\pm 5.89}$	21.03M (100%)	65.65M (100%)	21.01M (100%)
FedBABU	$41.41_{\pm 8.87}$	$88.38_{\pm 7.16}$	$19.50_{\pm 7.71}$	21.03M (100%)	65.65M (100%)	20.66M (98%)
FedRep	$44.42_{\pm 7.80}$	$92.63_{\pm 3.77}$	$23.67_{\pm 5.97}$	21.03M (100%)	65.65M (100%)	20.66M (98%)
Vanilla Attention	$44.63_{\pm 8.67}$	$88.90_{\pm 5.87}$	$22.55_{\pm 6.37}$	21.03M (100%)	65.65M (100%)	13.89M (66%)
FedPerfix (ours)	$\mathbf{48.10}_{\pm 7.76}$	$\mathbf{93.17}_{\pm 3.51}$	$\mathbf{24.38}_{\pm 8.47}$	24.42M (116%)	66.58M (101%)	20.66M (98%)

tion gathered from the global aggregation can alleviate the overfitting due to few training samples on the client, leading to higher performance.

In addition, it is worth noting that the methods that focus on the global objective instead of information transferring from the global to the client model, such as Per-FedAVG, achieve better performance than FedAVG but still show inferior performance compared to Local when facing extreme label skew or concept skew, *i.e.*, CIFAR-100 and Office-Home. This suggests that direct modification of the global objective may not always be the most effective way when using ViTs. Performing meta-learning on relatively small-size client data for a relatively large ViT is a challenging task, which limits the effectiveness of Per-FedAVG.

On the other hand, how to leverage the information provided in the global model is crucial for partial model personalization. The layer normalization version of FedBN shows little impact on the performance, leading to a negligible improvement based on FedAVG. However, APFL, FedRep, Vanilla Attention, FedBABU, and FedPerfix find a more suitable way to transfer the information from the global model, leading to superior performance than other methods. Consistent with the conclusion we draw from the empirical study, FedRep, FedBABU, and Vanilla Attention personalized the sensitive parts in ViT, thus outperforming FedAVG and Local by a significant margin. As an extension, APFL and our method, FedPerfix, balance the information between the local and global models in a proper manner, leading to the highest two performances considering the performance across all three datasets.

Surprisingly, we find that there is a connection between FedPerfix and APFL. Equation 3, which shows the output of a self-attention with Prefixes, can be rewritten as

$$\begin{aligned} head(x) =& (1-\lambda(x)) \underbrace{Attn(x\boldsymbol{W}_q, x\boldsymbol{W}_k, x\boldsymbol{W}_v)}_{\text{aggregated attention}} \\ &+ \lambda(x) \underbrace{Attn(x\boldsymbol{W}_q, x\boldsymbol{P}_k, x\boldsymbol{P}_v)}_{\text{personalized attention}} \end{aligned} \tag{6}$$

where $\lambda(x)$ is a scalar representing the normalized attention weights on the prefixes [10]. More details are provided in Supplementary B. It reveals that *the local prefixes are learned as a mixture coefficient between the aggregated attention from the global model and the personalized attention from the prefixes*. The high performance of both APFL and FedPerfix indicates the effectiveness of a mixture between the local and global models in PFL. Compared with APFL, FedPerfix only mixed the sensitive self-attention layers instead of the entire model, leading to higher performance. Meanwhile, FedPerfix trains the Prefix, which can be considered as the mixture coefficient, and the model simultaneously, while APFL needs separate training for the personalized model. Therefore, FedPerfix can achieve higher performance with fewer **storage**, **computation**, and **communication** resources. Furthermore, we provide a more detailed resource requirements analysis of each method in the following Section 4.4.

4.3. Client-wise Performance

In Fig. 3, we present a density plot depicting the performance gain of each method in comparison to Local. Notably, FedPerfix demonstrates an average performance gain of over 10%, surpassing all other methods. Moreover, FedPerfix outperforms all other methods by achieving up to 30% performance gains for certain clients. Conversely, some clients experience performance degradation under other methods, while FedPerfix ensures performance

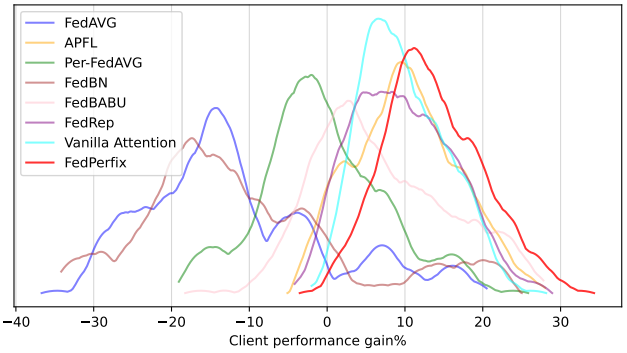


Figure 3. **Client-wise performance for each method on CIFAR-100.** Density of the accuracy gain compared with Local training is plotted. FedPerfix provides highest upper and lower bounds among all the methods.

gains for nearly all clients. In conclusion, FedPerfix shows a *higher upper bound* while maintaining *a high lower bound*, which will encourage more clients to involve in the federated training with performance-gaining guarantees for almost all clients.

4.4. Resource Requirements Analysis

In federated learning, it is crucial to consider the practicality of each method in terms of the storage, computation, and communication resources required by each participating client. As such, it is imperative to analyze the resource demands of each approach to determine its feasibility in realistic federated settings.

We present a resource analysis of the evaluated methods, taking into account the storage, computation, and communication demands, which are crucial aspects in practical federated learning scenarios where clients may have limited resources. Table 2 reports the parameter size required to store the model, the FLOPs needed for training, and the parameter size required for communication. Notably, FedPerfix achieves superior performance compared to other methods with slightly more resources for storage and computation but fewer resources for communication.

Overall, our proposed FedPerfix approach demonstrates the highest efficiency in terms of achieving superior performance while utilizing fewer storage, computation, and communication resources compared to other methods. These findings highlight the potential of our approach to effectively address the resource constraints often encountered in realistic federated learning scenarios.

5. Ablation Study

In this section, we conduct an ablation study on the CIFAR-100 dataset to evaluate the robustness of FedPerfix under various federated learning settings and to investigate FedPerfix with different designs.

Table 3. Performance on CIFAR-100 with different backbones.

Method	ResNet50	ViT-Small
FedAVG	$17.87_{\pm 13.16}$	$23.29_{\pm 11.31}$
Local	$28.34_{\pm 9.83}$	$34.74_{\pm 9.36}$
APFL	$30.71_{\pm 9.89}$	$44.88_{\pm 10.50}$
Per-FedAVG	$28.82_{\pm 8.53}$	$33.86_{\pm 8.01}$
FedBN	$19.33_{\pm 10.25}$	$23.94_{\pm 11.19}$
FedBABU	$24.56_{\pm 7.82}$	$41.41_{\pm 8.87}$
FedRep	$39.52_{\pm 10.92}$	$44.42_{\pm 7.80}$
FedPerfix	-	$48.10_{\pm 7.76}$

5.1. Performance Comparison with CNN Backbone

To further demonstrate the motivation behind our work, *i.e.*, addressing PFL with ViT, we conduct experiments to compare ViT and CNN as different backbones for the same set of methods. Given that the comparison methods are *originally designed with a CNN backbone*, we conduct experiments on CIFAR-100 with a CNN backbone as a reference to investigate the impact of replacing the CNNs with ViTs in these methods. To fairly compare the performance, we choose ResNet50 [11] as the CNN backbone due to its similar parameter size (**24.37M**) to ViT-Small [8] (**21.03M**).

The result is shown in Table 3. Simply replacing the CNN backbone with the ViT backbone significantly improves performance for each method without additional operations. It suggests that the advantages of ViTs that demonstrated superior performance compared to CNNs in centralized settings may also translate to personalized federated learning scenarios.

The success of ViTs in existing methods highlights their potential for personalized federated learning. However, there is a lack of approaches specifically designed for ViTs. To address this gap, we propose FedPerfix, which is tailored for ViTs and focuses on personalizing the sensitive parts of the network. By doing so, FedPerfix further improves performance compared to existing methods that incorporate ViTs. Our approach fills an important research gap and suggests that there is still much to be gained from exploring the unique properties of ViTs for personalized federated learning.

5.2. Different FL Settings

In order to assess the robustness of FedPerfix in more challenging federated learning scenarios, we conduct experiments that involve varying the number of clients with fewer samples per client and a lower client participation rate. We also compare the performance of FedPerfix with other competitive methods and report the results in Table 4. Our experimental results demonstrate that FedPerfix exhibits robust performance in different federated learning settings,

Table 4. Performance on CIFAR-100 under different federated learning settings.

Method	N = 64		N = 128	
	$r = 6.25\%$	$r = 12.5\%$	$r = 6.25\%$	$r = 12.5\%$
FedAvg	$19.76_{\pm 9.35}$	$23.29_{\pm 11.31}$	$19.64_{\pm 10.01}$	$23.29_{\pm 11.81}$
Local	$32.35_{\pm 11.64}$	$34.74_{\pm 9.36}$	$31.79_{\pm 12.00}$	$36.03_{\pm 9.57}$
APFL	$42.55_{\pm 13.51}$	$44.88_{\pm 10.50}$	$41.64_{\pm 12.77}$	$43.92_{\pm 9.40}$
FedRep	$35.72_{\pm 10.33}$	$44.42_{\pm 7.80}$	$35.88_{\pm 9.59}$	$44.70_{\pm 8.40}$
FedPerfix	$\mathbf{43.80}_{\pm 11.56}$	$\mathbf{48.10}_{\pm 7.76}$	$\mathbf{43.96}_{\pm 10.42}$	$\mathbf{46.96}_{\pm 9.00}$

outperforming all other methods consistently. Thus, FedPerfix can be considered a reliable solution for extreme federated learning scenarios.

5.3. Investigation of FedPerfix

In this subsection, we investigate the impact of the model size of FedPerfix and verify its effectiveness compared with other designs.

Impact of Model Size. The model size is a crucial factor in federated learning, as it determines the resource requirements for storage, computation, and communication. To examine the impact of model size on the performance of FedPerfix, we conducted experiments with three different ViT backbones: ViT-Tiny, ViT-Small (Default), and ViT-Base. Additionally, we included the results of the baselines and two competitive methods as a reference. As shown in Table 5, FedPerfix still yields superior performance with different model sizes. Besides, the larger models generally performed better, with ViT-Base achieving the highest performance for each method. However, we note that the performance improvement decreases as the model size increases, suggesting a potential trade-off between model size and performance. This finding highlights the importance of selecting an appropriate model size to achieve the optimal balance between performance and resource requirements, particularly in resource-constrained environments where storage and computation resources are limited.

FedPerfix vs. Vanilla Prefix-tuning. As mentioned in Section 3.5, the initialization of the Prefixes will affect the performance. To show the effectiveness of the parallel attention design in FedPerfix, we conduct experiments with Vanilla Prefix-tuning under two different initialization, initialized with zero (Prefix-Z) and random initialization (Prefix-R). Then, we compare the results with FedPerfix. The result is shown in Table 6. As expected, the prefixes generated from the parallel attention yield better performance than the other two manually initialized prefixes, highlighting the effectiveness of the parallel attention design in FedPerfix.

FedPerfix vs. Prompts & Adapters. Prompts and adapters are also popular plugins that are widely used. To verify whether FedPerfix is the most effective among all families of the plugins, we also apply Prompts and Adapters to personalized federated learning. Specifically, we append several trainable Prompts to the input embeddings and keep them updated locally as the implementation of Prompt-tuning [15]. Meanwhile, we add Adapters to the MLP layers as the implementation of Adatper-tuning [30]. The results are shown in Table 6. Adding personalized Prompts doesn't improve the performance compared with simply keeping the classification head updated locally, indicating that the transformation to the input is not effective in PFL. However, adding Adapters to the MLP layers can also lead to a promising result as FedPerfix. Recap the result of our empirical study shown in Table 1, MLP layers are also sensitive to data distribution in a ViT. We note that as another instance to show the effectiveness of adding plugins to the sensitive parts of a ViT in PFL.

Table 5. Performance on CIFAR-100 with different model sizes.

Method	ViT-Tiny	ViT-Small	ViT-Base
FedAVG	$19.56_{\pm 10.27}$	$23.29_{\pm 11.31}$	$24.82_{\pm 11.53}$
Local	$33.73_{\pm 9.20}$	$34.74_{\pm 9.36}$	$38.93_{\pm 10.08}$
APFL	$37.66_{\pm 10.04}$	$44.88_{\pm 10.50}$	$47.57_{\pm 9.57}$
FedRep	$41.58_{\pm 7.19}$	$44.42_{\pm 7.80}$	$44.34_{\pm 8.35}$
FedPerfix	$\mathbf{44.71}_{\pm 8.47}$	$\mathbf{48.10}_{\pm 7.76}$	$\mathbf{48.40}_{\pm 8.18}$

Table 6. Performance of different designs on CIFAR-100. Prefix-Z and Prefix-R mean vanilla prefix-tuning with zero and random initialization.

FedPerfix	Prefix-Z	Prefix-R	Prompts	Adapters
$\mathbf{48.10}_{\pm 7.76}$	$47.37_{\pm 8.47}$	$46.98_{\pm 8.10}$	$44.19_{\pm 8.13}$	$47.99_{\pm 8.59}$

6. Conclusion

In this work, we studied two research questions of where and how to personalize a ViT in federated learning. We conducted an empirical study to reveal that the self-attention and classification layers are the most sensitive layers for personalization. Based on that, we proposed FedPerfix, a novel method that introduces Prefixes with parallel attention to personalize the self-attention layers. Through extensive experiments on various datasets with different degrees of data heterogeneity, we demonstrated that FedPerfix achieves state-of-the-art performance while also reducing resource requirements. Our work focuses on ViTs and represents a shift in attention from the extensively researched CNNs, serving as inspiration for further investigation.

7. Acknowledgement

This work is partially supported by the NSF/Intel Partnership on MLWiNS under Grant No. 2003198.

References

[1] Manoj Ghuhan Arivazhagan, Vinay Aggarwal, Aaditya Kumar Singh, and Sunav Choudhary. Federated Learning with Personalization Layers, Dec. 2019. arXiv:1912.00818 [cs, stat]. 1, 3

[2] Han Cai, Chuang Gan, Ligeng Zhu, and Song Han. TinyTL: Reduce Memory, Not Parameters for Efficient On-Device Learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 11285–11297. Curran Associates, Inc., 2020. 2

[3] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-End Object Detection with Transformers, May 2020. arXiv:2005.12872 [cs]. 1

[4] Hong-You Chen and Wei-Lun Chao. ON BRIDGING GENERIC AND PERSONALIZED FEDERATED LEARNING FOR IMAGE CLASSIFICATION. 2022. 3

[5] Liam Collins, Hamed Hassani, Aryan Mokhtari, and Sanjay Shakkottai. Exploiting Shared Representations for Personalized Federated Learning. In *Proceedings of the 38th International Conference on Machine Learning*, pages 2089–2099. PMLR, July 2021. ISSN: 2640-3498. 1, 2, 3

[6] Yuyang Deng, Mohammad Mahdi Kamani, and Mehrdad Mahdavi. Adaptive Personalized Federated Learning, Nov. 2020. arXiv:2003.13461 [cs, stat]. 1, 3, 6

[7] Canh T. Dinh, Nguyen H. Tran, and Tuan Dung Nguyen. Personalized Federated Learning with Moreau Envelopes, Jan. 2022. arXiv:2006.08848 [cs, stat]. 3

[8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, June 2021. arXiv:2010.11929 [cs]. 1, 3, 6, 8

[9] Alireza Fallah, Aryan Mokhtari, and Asuman Ozdaglar. Personalized Federated Learning: A Meta-Learning Approach, Oct. 2020. arXiv:2002.07948 [cs, math, stat]. 1, 3, 6

[10] Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. Towards a Unified View of Parameter-Efficient Transfer Learning. Feb. 2022. 3, 7

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 8

[12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. June 2021. 2

[13] Yutao Huang, Lingyang Chu, Zirui Zhou, Lanjun Wang, Jiangchuan Liu, Jian Pei, and Yong Zhang. Personalized Cross-Silo Federated Learning on Non-IID Data. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(9):7865–7873, May 2021. Number: 9. 2

[14] Sergey Ioffe and Christian Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, Mar. 2015. arXiv:1502.03167 [cs]. 4

[15] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual Prompt Tuning, July 2022. arXiv:2203.12119 [cs]. 2, 3, 9

[16] Alex Krizhevsky. Learning Multiple Layers of Features from Tiny Images. *University of Toronto*, 2012. 6

[17] Tian Li, Shengyuan Hu, Ahmad Beirami, and Virginia Smith. Ditto: Fair and Robust Federated Learning Through Personalization. In *Proceedings of the 38th International Conference on Machine Learning*, pages 6357–6368. PMLR, July 2021. ISSN: 2640-3498. 2

[18] Xiaoxiao Li, Meirui Jiang, Xiaofei Zhang, Michael Kamp, and Qi Dou. FedBN: Federated Learning on Non-IID Features via Local Batch Normalization, May 2021. arXiv:2102.07623 [cs]. 1, 2, 3, 4

[19] Xin-Chun Li, De-Chuan Zhan, Yunfeng Shao, Bingshuai Li, and Shaoming Song. FedPHP: Federated Personalization with Inherited Private Models. In Nuria Oliver, Fernando Pérez-Cruz, Stefan Kramer, Jesse Read, and Jose A. Lozano, editors, *Machine Learning and Knowledge Discovery in Databases. Research Track*, Lecture Notes in Computer Science, pages 587–602, Cham, 2021. Springer International Publishing. 2

[20] Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation, Jan. 2021. arXiv:2101.00190 [cs]. 2, 3

[21] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows, Aug. 2021. arXiv:2103.14030 [cs]. 3

[22] Ze Liu, Jia Ning, Yue Cao, Yixuan Wei, Zheng Zhang, Stephen Lin, and Han Hu. Video Swin Transformer. *arXiv:2106.13230 [cs]*, June 2021. arXiv: 2106.13230. 3

[23] Jun Luo, Matias Mendieta, Chen Chen, and Shandong Wu. PGFed: Personalize Each Client's Global Objective for Federated Learning, Dec. 2022. arXiv:2212.01448 [cs]. 1

[24] Yishay Mansour, Mehryar Mohri, Jae Ro, and Ananda Theertha Suresh. Three Approaches for Personalization with Applications to Federated Learning, July 2020. arXiv:2002.10619 [cs, stat]. 2

[25] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data, Feb. 2017. arXiv:1602.05629 [cs]. 1

[26] Matias Mendieta, Taojiannan Yang, Pu Wang, Minwoo Lee, Zhengming Ding, and Chen Chen. Local Learning Matters: Rethinking Data Heterogeneity in Federated Learning, Apr. 2022. arXiv:2111.14213 [cs]. 2

[27] Jaehoon Oh, Sangmook Kim, and Se-Young Yun. FedBABU: Towards Enhanced Representation for Federated Image Classification, Mar. 2022. arXiv:2106.06042 [cs]. 1, 2, 3, 4

[28] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner,

Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. 6

[29] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. AdapterFusion: Non-Destructive Task Composition for Transfer Learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 487–503, Online, 2021. Association for Computational Linguistics. 2

[30] Jonas Pfeiffer, Andreas Rücklé, Clifton Poth, Aishwarya Kamath, Ivan Vulić, Sebastian Ruder, Kyunghyun Cho, and Iryna Gurevych. AdapterHub: A Framework for Adapting Transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 46–54, Online, 2020. Association for Computational Linguistics. 2, 3, 9

[31] Krishna Pillutla, Kshitiz Malik, Abdel-Rahman Mohamed, Mike Rabbat, Maziar Sanjabi, and Lin Xiao. Federated Learning with Partial Model Personalization. In *Proceedings of the 39th International Conference on Machine Learning*, pages 17716–17758. PMLR, June 2022. ISSN: 2640-3498. 1

[32] Liangqiong Qu, Yuyin Zhou, Paul Pu Liang, Yingda Xia, Feifei Wang, Ehsan Adeli, Li Fei-Fei, and Daniel Rubin. Rethinking Architecture Design for Tackling Data Heterogeneity in Federated Learning, Apr. 2022. arXiv:2106.06047 [cs]. 2, 3

[33] Sebastian Ruder. An overview of gradient descent optimization algorithms, June 2017. arXiv:1609.04747 [cs]. 6

[34] Aviv Shamsian, Aviv Navon, Ethan Fetaya, and Gal Chechik. Personalized Federated Learning using Hypernetworks, Mar. 2021. arXiv:2103.04628 [cs]. 3

[35] Guangyu Sun, Matias Mendieta, Taojiannan Yang, and Chen Chen. Conquering the Communication Constraints to Enable Large Pre-Trained Models in Federated Learning, Nov. 2022. arXiv:2210.01708 [cs]. 3

[36] Yue Tan, Guodong Long, Lu Liu, Tianyi Zhou, Qinghua Lu, Jing Jiang, and Chengqi Zhang. FedProto: Federated Prototype Learning across Heterogeneous Clients. *Proceedings of the AAAI Conference on Artificial Intelligence*, 36(8):8432–8440, June 2022. Number: 8. 2, 3

[37] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Herve Jegou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, volume 139, pages 10347–10357, July 2021. 1

[38] Hemanth Venkateswara, Jose Eusebio, Shayok Chakraborty, and Sethuraman Panchanathan. Deep hashing network for unsupervised domain adaptation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5018–5027, 2017. 2, 6

[39] Ross Wightman. PyTorch Image Models, 2019. Publication Title: GitHub repository. 6

[40] Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, and Bingbing Ni. MedMNIST v2-A large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10(1):41, 2023. Publisher: Nature Publishing Group UK London. 2, 6

[41] Bruce X. B. Yu, Jianlong Chang, Lingbo Liu, Qi Tian, and Chang Wen Chen. Towards a Unified View on Visual Parameter-Efficient Transfer Learning, Oct. 2022. arXiv:2210.00788 [cs]. 3, 5

[42] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection, Apr. 2022. arXiv:2203.03605 [cs] version: 3. 1

[43] Michael Zhang, Karan Sapra, Sanja Fidler, Serena Yeung, and Jose M. Alvarez. Personalized Federated Learning with First Order Model Optimization. Jan. 2023. 2